

ROBOTICS · LESSON 3

Reading the World

Swap the knob for an ultrasonic sensor — the arm tracks distance, and the loop runs without your hand. The first taste of autonomy.

ROBO-003 · ~35 MIN · BEGINNER

1. The one idea

The input becomes the **world**. Swap the knob for an HC-SR04 ultrasonic sensor, and the servo arm now tracks *distance* instead of your hand. The loop runs without you — the first taste of autonomy.

2. Why it matters

A robot that only does what your hand says isn't autonomous. The moment a sensor drives the loop, the machine responds to the world on its own — move your hand and the arm follows the gap, not your fingers. Every autonomous behavior starts exactly here: replace the human input with a measurement, and the same `read → process → actuate` loop suddenly belongs to the robot.

3. Parts & wiring

- ESP32 dev board
- SG90 servo — signal on **GPIO13**, power on external 5V (as in Lesson 1)
- HC-SR04 ultrasonic sensor
- Two resistors for a voltage divider on ECHO (e.g. 1kΩ + 2kΩ)

Sensor pin	To	Why
VCC	5V	the HC-SR04 is a 5V part
GND	GND — shared with the ESP32	common reference
TRIG	GPIO23	a 10 μs pulse here fires a ping; 3.3V is enough to trigger it
ECHO	GPIO22 through a divider	ECHO drives 5V — straight into a 3.3V GPIO it can damage the pin

The ECHO divider is the wire beginners skip. The HC-SR04's ECHO line swings to 5V, but ESP32 GPIOs are 3.3V parts. Drop it with a divider (1k Ω from ECHO to the pin, 2k Ω from the pin to GND gives ~3.3V). A 3.3V-native module (RCWL-1601) skips this — but on a classic HC-SR04, the divider protects your board.

4. Predict (do this before uploading)

Map 2–40 cm onto 0–180°, then guess:

- Where is the arm when your hand is 20 cm away?
- Which way does it swing as your hand gets *closer*?
- What happens when nothing is in range — where does the arm go?

Write it down. §6 checks it.

5. Build it — time, not voltage

```
#include <ESP32Servo.h>

Servo armServo;
const int SERVO_PIN = 13;
const int TRIG_PIN = 23;
const int ECHO_PIN = 22;    // 5V echo → 3.3V via a divider

void setup() {
  Serial.begin(115200);
  armServo.attach(SERVO_PIN);
  pinMode(TRIG_PIN, OUTPUT);
  pinMode(ECHO_PIN, INPUT);
}

long readDistanceCm() {
  digitalWrite(TRIG_PIN, LOW); delayMicroseconds(2);
  digitalWrite(TRIG_PIN, HIGH); delayMicroseconds(10);    // 10 us trigger pulse
  digitalWrite(TRIG_PIN, LOW);
  long us = pulseIn(ECHO_PIN, HIGH, 30000);                // echo width, 30 ms timeout
  if (us == 0) return -1;                                   // out of range / no echo
  return us / 58;                                           // round-trip us → cm
}

void loop() {
  long cm = readDistanceCm();
  if (cm > 0) {
    int angle = map(constrain(cm, 2, 40), 2, 40, 0, 180);
    armServo.write(angle);
    Serial.printf("dist=%3ld cm  angle=%3d\n", cm, angle);
  } else {
    Serial.println("out of range");
  }
  delay(60);    // give the sensor time between pings
}
```

The new skill is that the sensor speaks in **time**, not voltage. The ECHO line stays HIGH for exactly as long as the sound takes to fly out and bounce back; `pulseIn` measures that width in microseconds,

and distance is that time divided by 58 (the round-trip μs per centimetre). Naming it — *the sensor measures time, and you turn time into distance* — is the whole lesson.

6. Test against your prediction

Open the Serial Monitor at 115200 baud and wave your hand toward and away from the sensor. The arm sweeps with the distance — no knob anywhere. Did the extremes match your \$4 guesses? Move your hand out of range and watch it report `out of range` instead of lying about a distance.

7. What goes wrong (pre-loaded debugging)

- **ESP32 acts erratic or the pin dies** → you wired 5V ECHO straight to a GPIO. Add the divider; protect the 3.3V pin.
- **Always reads out of range / 0** → TRIG and ECHO swapped, nothing in front, or the target is past ~4 m (the echo times out).
- **Distance jumps around** → ultrasonic is noisy and pinging too fast lets echoes overlap. Keep ≥ 60 ms between pings, and average a few reads if you need it steady.
- **Arm slams to an extreme with nothing in front** → an out-of-range reading mapped to a wild angle. Handle the bad reading (return -1 and skip) instead of trusting it — the seed of the fail-safe you'll need in Lesson 5.

8. Checkpoint

The loop now runs on a measurement, not your hand. You turned a time-of-flight echo into a distance, and a distance into motion — **sense** → **act**, autonomously. That is the seed of every autonomous robot: a sensor where the human used to be.

9. Push further

- Reuse the Lesson 2 guard: refuse to move on an out-of-range reading (a lost-signal safe pose).
- Mount the sensor on a second servo and sweep it — a scanning rangefinder.
- This sensor becomes the rover's **eyes** in Lesson 5.